

Balanced Binary Tree

Difficulty: Easy

Patterns: Tree, Binary Tree, Depth-First Search, Recursion

LeetCode: <https://leetcode.com/problems/balanced-binary-tree/>

1. Problem Summary

Given the `root` of a binary tree, determine whether the tree is height-balanced.

A binary tree is height-balanced when, for every node, the height difference between its left subtree and right subtree is at most 1.

Examples:

- `root = [3,9,20,null,null,15,7]` returns `true`
- `root = [1,2,2,3,3,null,null,4,4]` returns `false`
- `root = []` returns `true`

2. Constraints and Observations

- Constraint: The number of nodes is in `[0, 5000]`.
- What it implies: A linear `O(n)` solution is ideal. A repeated-height solution can degrade to `O(n^2)` on skewed trees.
- Constraint: `-10^4 <= Node.val <= 10^4`.
- What it implies: Node values do not matter. Only the shape of the tree matters.
- Observation: Balance is a local condition at every node, but it depends on subtree heights.
- Observation: If a subtree is already unbalanced, the whole tree is unbalanced. We can stop caring about exact heights above that subtree.

3. Pattern Recognition

This is a binary tree DFS problem.

The natural helper is a function that computes the height of a subtree. The catch is that checking balance also needs those same heights. If we compute height separately for every node, we repeat work.

The optimal pattern is postorder DFS:

1. Solve the left subtree.
2. Solve the right subtree.
3. Use both answers to decide the current node.

This is postorder because a parent cannot know whether it is balanced until both child subtree heights are known.

4. Naive Approach

The first natural idea is:

1. For each node, compute the height of its left subtree.
2. Compute the height of its right subtree.
3. Check whether their difference is at most 1.
4. Recursively check whether the left and right subtrees are balanced.

This is correct, but it repeats height calculations. In a skewed tree, the height of the same long chain may be recomputed many times.

Worst-case complexity:

- Time: $O(n^2)$ for a highly skewed tree.
- Space: $O(h)$ recursion stack, where h is the height of the tree.

The bottleneck is repeated subtree-height computation.

5. Key Intuition

When a DFS call finishes a subtree, it can report two things:

1. The subtree height.
2. Whether the subtree is balanced.

We can combine those into one return value:

- Return the real height if the subtree is balanced.
- Return `-1` as a sentinel if the subtree is unbalanced.

That means every node is processed once. The parent only needs to check:

- Did either child return `-1`?
- Are the left and right heights more than `1` apart?

If either answer is yes, the parent returns `-1` too.

6. Optimal Solutions Ranked

Variant 1: Top-down Height Check

- Idea: For every node, compute `height(left)`, compute `height(right)`, then recursively validate both children.
- Time: $O(n^2)$ worst case, $O(n \log n)$ on balanced trees.
- Space: $O(h)$ recursion stack.
- Tradeoff: Easy to derive, but repeated height work makes it weaker for skewed trees.

Variant 2: Bottom-up DFS Returning `(balanced, height)`

- Idea: Each recursive call returns both whether the subtree is balanced and its height.
- Time: $O(n)$.
- Space: $O(h)$ recursion stack.
- Tradeoff: Very explicit and readable. Slightly more verbose because it returns a tuple.

Variant 3: Bottom-up DFS With Sentinel Height

- Idea: Each call returns a valid height, or `-1` if that subtree is already unbalanced.
- Time: $O(n)$.
- Space: $O(h)$ recursion stack.
- Tradeoff: Compact and efficient. The sentinel makes early failure simple.

Most optimal choice: bottom-up DFS with sentinel height.

No algorithm can improve the worst-case time below $O(n)$ because a fully balanced tree may require checking every node before we can safely return `true`. The best we can do is visit each node once and stop early when an imbalance is found.

7. Most Optimal Approach

Use a recursive helper called `height`.

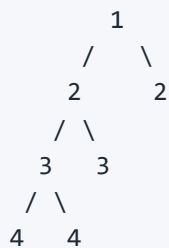
For any node:

1. If the node is `None`, its height is `0`.
2. Recursively compute the left height.
3. If the left subtree returned `-1`, return `-1`.
4. Recursively compute the right height.
5. If the right subtree returned `-1`, return `-1`.
6. If `abs(left_height - right_height) > 1`, return `-1`.
7. Otherwise return `1 + max(left_height, right_height)`.

The final answer is `height(root) != -1`.

8. Algorithm

1. Define `height(node)`.
2. If `node` is `None`, return `0`.
3. Recursively get `left_height = height(node.left)`.
4. If `left_height == -1`, return `-1`.
5. Recursively get `right_height = height(node.right)`.
6. If `right_height == -1`, return `-1`.
7. If the absolute difference between the two heights is greater than `1`, return `-1`.
8. Return `1 + max(left_height, right_height)`.
9. Return whether `height(root)` is not `-1`.



Bottom-up returns:

Node	Left height	Right height	Result
4	0	0	balanced, return 1
4	0	0	balanced, return 1
3 with children 4,4	1	1	balanced, return 2
3 leaf	0	0	balanced, return 1
left 2	2	1	balanced, return 3
right 2	0	0	balanced, return 1
root 1	3	1	difference is 2 , return -1

Since the root returns -1 , the final answer is false .

11. Correctness Argument

Invariant:

For every call `height(node)` , the function returns:

- the exact height of the subtree rooted at `node` if that subtree is balanced;
- -1 if that subtree is not balanced.

Why the invariant holds:

- Base case: An empty subtree has height 0 and is balanced, so returning 0 is correct.
- Recursive step: Assume the invariant is true for the left and right children.

- If either child returns `-1`, that child subtree is unbalanced, so the current subtree is also unbalanced. Returning `-1` is correct.
- Otherwise, both returned values are exact child heights. The current subtree is balanced exactly when their difference is at most `1`.
- If the difference is greater than `1`, returning `-1` is correct.
- If the difference is at most `1`, the current subtree is balanced and its height is `1 + max(left_height, right_height)`.

Therefore, `height(root) != -1` exactly when the entire tree is balanced.

12. Complexity Analysis

- Time: $O(n)$, where n is the number of nodes.
- Why: Each node is visited at most once, and each visit does $O(1)$ work besides recursive calls.
- Space: $O(h)$ for the recursion stack, where h is the height of the tree.
- Worst-case stack space: $O(n)$ for a skewed tree.
- Best-case stack space: $O(\log n)$ for a balanced tree.

13. Edge Cases

- Empty tree: `root = None`, return `True`.
- Single node: both child heights are `0`, return `True`.
- Perfectly balanced tree: all nodes have left/right height difference `0` or `1`, return `True`.
- Root-level imbalance: child subtrees are individually balanced but root height difference is greater than `1`, return `False`.
- Deep skewed tree: the algorithm is still linear, but recursion depth can become large in Python.
- Negative node values: no effect, because values are ignored.

14. Approach Code: Explicit Tuple Return

```
from typing import Optional

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def isBalanced(self, root: Optional[TreeNode]) -> bool:
        def dfs(node: Optional[TreeNode]) -> tuple[bool, int]:
            if node is None:
                return True, 0

            left_balanced, left_height = dfs(node.left)
            right_balanced, right_height = dfs(node.right)

            balanced = (
                left_balanced
                and right_balanced
                and abs(left_height - right_height) <= 1
            )

            height = 1 + max(left_height, right_height)
            return balanced, height

        balanced, _ = dfs(root)
        return balanced
```


15. Final Clean Code

```
from typing import Optional

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def isBalanced(self, root: Optional[TreeNode]) -> bool:
        def height(node: Optional[TreeNode]) -> int:
            if node is None:
                return 0

            left_height = height(node.left)
            if left_height == -1:
                return -1

            right_height = height(node.right)
            if right_height == -1:
                return -1

            if abs(left_height - right_height) > 1:
                return -1

            return 1 + max(left_height, right_height)

        return height(root) != -1
```

16. Key Takeaways

- Main pattern: postorder DFS on a binary tree.
- Main bottleneck: recomputing subtree heights from scratch at every node.
- Main trick: return height for balanced subtrees and `-1` as an unbalanced sentinel.
- Interview explanation: "I compute heights bottom-up. If any subtree is unbalanced, I propagate `-1` upward and the whole tree fails."